

Requerimientos Técnicos a Evaluar

Trabajo Práctico Final — App World Cup 2026

Arquitectura y separación de capas

- El frontend (WorldCupApp) no realiza llamadas directas a la WorldCupAPI — toda comunicación pasa por el WorldCupCore
 - El componente Angular no contiene lógica de negocio — solo presenta datos y captura interacciones del usuario
 - El service Angular no contiene lógica de negocio — solo adapta la respuesta del backend al modelo de vista
 - El controller NestJS no contiene lógica de negocio — solo recibe el request y delega al service
 - El service NestJS centraliza toda la lógica: llamadas a la WorldCupAPI, transformación de datos y construcción de la respuesta
-

Frontend — HTML

- El template usa etiquetas HTML semánticas donde corresponde (`<section>`, `<article>`, `<header>`, `<nav>`, `<main>`, etc.)
 - No se usan etiquetas de presentación obsoletas (`<font>`, `<center>`, `<b>` en lugar de `<strong>`, etc.)
 - Las listas se marcan con `<ul>` / `<ol>` + `<li>`, no con `<div>` consecutivos
 - Los formularios y controles interactivos usan las etiquetas correspondientes (`<form>`, `<label>`, `<input>`, `<select>`, `<button>`)
-

Frontend — CSS

- Se prioriza el uso de variables CSS globales del proyecto para identidad visual (`--team-primary`, `--team-secondary`, etc.); se permiten estilos locales por componente cuando sea necesario. No se usan estilos inline en el template salvo casos justificados
 - Alcance de visualización: desktop-first. Debe verse correctamente en resoluciones de monitor comunes.
 - El layout se resuelve con Flexbox o CSS Grid — no con tablas ni posicionamiento absoluto
-

Frontend — Angular

- La identidad visual de la aplicación es consistente en todas las pantallas del proyecto
 - Cada feature tiene su propio componente (`.ts` + `.html` + `.css`) y su propio service
 - El componente maneja los cuatro estados obligatorios: cargando, error, sin datos y datos disponibles
 - El template usa directivas estructurales correctamente (`*ngIf`, `*ngFor`)
 - El binding es el adecuado según el caso: interpolación, property binding, event binding o two-way binding
 - Los modelos están tipados con interfaces: `api.interface.ts` para el modelo de la API y `view-model.interface.ts` para el modelo de vista
 - El service Angular se suscribe a los Observables del `HttpClient` y maneja correctamente los casos de éxito y error
 - Los services en angular no contienen reglas de dominio; solo orquestan llamadas HTTP al core, manejan estado de vista y adaptan los datos al modelo de vista.
 - La carga de datos inicial se realiza en `ngOnInit`, no en el constructor
-

Backend — NestJS

- Cada feature tiene su propio controller y su propio service, organizados en el directorio de su sección
 - Los endpoints del WorldCupCore siguen las convenciones REST: sustantivos en plural, verbos HTTP correctos, códigos de estado apropiados (200, 201, 400, 404, 500, etc.)
 - Los parámetros de entrada están tipados con DTOs
 - Los parámetros de entrada HTTP deben estar tipados con DTOs (Requests) en los controllers. En services no es obligatorio usar DTO, pero sí tipos explícitos.
 - Los errores de la WorldCupAPI son capturados y convertidos en respuestas estructuradas con un `messageCode` específico — nunca se propaga el error crudo al frontend
 - El `teamId` y el `lang` se leen de las variables de entorno (`.env`) — no están hardcodeados en el código
 - El parámetro `?lang=` se envía correctamente en todas las llamadas a la WorldCupAPI que lo soporten
 - Cada controller y su request correspondiente deben estar documentados utilizando swagger.
-

Calidad de código

- El código TypeScript está tipado: no se usan `any` salvo casos justificados

- Los nombres de variables, funciones y clases son descriptivos y en inglés
- No hay código comentado ni `console.log` de depuración en la entrega final
- No se deben modificar las funcionalidades provistas como ejemplo (Plantel, Historia del Equipo, Simular Final)

Otros Requisitos

- El repositorio incluye un README con instrucciones de instalación, que integrante del equipo es el tech leader y la tabla de integrantes con su funcionalidad asignada.
 - No se debe modificar la World Cup API - Usar una World Cup API modificada será motivo de desaprobación del trabajo práctico.
 - El proyecto compila y corre sin errores con `npm install + npm run start`
-